

Student No._____ **Name:**_____

특별한 언급이 없는 한, 32bit x86 프로세서의 protected mode 에서 MASM 을 사용함을 가정함

1.1 [5pt] 10 진수 소수인 0. 090625 가 아래 MASM 프로그램의 num 에 저장된다 가정하자. 메모리를 표현하고 있는 오른쪽 그림에서 num 이 오프셋 100 에 위치한다고 가정했을 때, 그림상의 메모리의 각 셀에 들어갈 값을 16 진수로 채워넣으시오.

The diagram illustrates the memory layout for a variable named `num` of type `REAL4`. On the left, the assembly code is shown: `.data`, `num REAL4 ?`. On the right, a vertical stack of 10 memory slots is shown, labeled `memory`. The first slot is pointed to by an arrow labeled `offset 100`, and the last slot is pointed to by an arrow labeled `offset 108`, indicating a total size of 8 bytes.

1.2 [5pt] 부동소수점 수 num 에 16 진수 00700000 이 저장되어 있다 가정하자. num 에 저장된 값을 다음 표현법을 이용해 표현하시오 (단, 빈 칸에는 10 진수로 적으시오).

$$\boxed{} \times 2 = \boxed{}$$

2. [5pt] 다음은 MASM 으로 작성된 프로그램의 메모리 관리에 관한 설명이다. 옳은 설명을 모두 고르시오.

- ① 프로시저의 지역변수들을 위한 메모리 공간은 STACK 세그먼트 내에 할당된다.
- ② 동적 메모리 할당을 운영체제에게 요청하면 요청받은 크기만큼의 메모리 블록이 HEAP 메모리 영역에 할당되어지고 그 주소가 반환된다.
- ③ 프로시저에서 동적 할당한 메모리는 프로시저 종료 시 return 과정에서 ESP 및 EBP 의 조정에 의해 자동 반환된다.
- ④ 세그먼트의 base 주소와 offset 을 더하면 physical 주소를 얻을 수 있다.
- ⑤ 접근 요청을 받은 메모리 상의 데이터가 cache 에 없을 때, page fault 가 발생한다.

3.(1) [10pt] 주어진 두 인자를 뒤바꾸는 swap 함수가 다음과 같이 c 언어로 쓰여져 있다. swap 함수를 어셈블리어로 다시 작성하시오. 단, C 의 calling convention 이 아닌, STDCALL 을 사용한다 가정하고 빈칸을 채워 프로시저를 작성하시오. (enter 와 leave 는 사용할 수 없음)

C 로 작성된 swap 함수	어셈블리어로 작성한 swap 프로시저
<pre>void swap(int * a, int * b){ int temp = *a; *a = *b; *b = temp; }</pre>	<pre>swap PROC </pre> <pre>swap ENDP</pre>

3.(2) [5pt] 문제 3.(1)의 swap 프로시저를 STDCALL calling convention 을 사용하여 호출하고자 한다. Swap 프로시저가 어떻게 호출되어야 할지 빈 칸을 채우시오. (invoke를 사용하여 swap 프로시저를 호출할 수 없음)

swap 함수를 호출하는 부분	swap 프로시저를 호출하는 부분
<pre>int x = 1; int y = 2; void main(void){ ... swap(&x, &y); ... }</pre>	<pre>.data x DWORD 1 y DWORD 2 .code swap PROC ... swap ENDP main PROC ... </pre> <pre>... main ENDP END main</pre>

Student No. _____ Name: _____

4. [15pt] 다음 조건부 점프 명령어들의 실행 조건을 Flag 비트들의 관계 또는 값으로 표현하시오. (필요시 AND/OR 등을 사용할 수 있음. 예시: $PF = 1 \text{ AND } ZF = PF$)

	CF	OF	PF	SF	ZF
JA					
JB					
JE					
JG					
JL					
JNE					

5. 다음 어셈블리어 프로그램들이 실행된다면, 프로그램이 수행된 직후 각 문제에서 요구되는 값이 무엇인지 **16 진수**로 답하시오. 만약 실행되지 않고, 어셈블 과정, 또는 실행과정에서 에러가 발생한다면, 어느 구문에서 어떤 에러가 발생하는지 답하시오.

(1) [5pt]

```
MOV EAX, -50
MOV ECX, -1
L1: INC AL
    LOOPNE L1
```

답: EAX 레지스터에 저장될 값을 16 진수로

(2) [5pt]

```
MOV AL, -130
MOV BL, +130
MOV CL, -32
MOV DL, +16
SUB AL, BL
MOV CL, DL
JG L1
ADD DL, CL
L1:
```

답: DL 레지스터에 저장될 값을 16 진수로

(3) [5pt]

(WriteChar 는 AL 레지스터에 저장된 문자를 화면에 출력하는 irvine32 라이브러리임)

```
include d:/irvine32/irvine32.inc
```

```
.386
```

```
.model flat, stdcall
```

```
.stack 4096
```

```
ExitProcess PROTO, dwExitCode:DWORD
```

```
.code
```

```
Message PROC
```

```
    MOV ECX, 10
```

```
L1:    MOV AL, '*'      ;ascii code 2A
```

```
    CALL WriteChar
```

```
    LOOP L1
```

```
    RET
```

```
Message ENDP
```

```
MAIN PROC
```

```
    MOV AH, 0
```

```
    MOV AL, -1
```

```
    MOV CL, 8
```

```
    CALL Message
```

```
    DIV CL
```

```
    INVOKE ExitProcess, 0
```

```
MAIN ENDP
```

```
END main
```

답: 화면에 표시되는 문자

6. 주어진 설명에 맞게 동작하도록 빈칸에 적절한 명령어들을 넣으시오. (박스당 명령어 1개)

(1) [10pt] 구구단(Multiplication table)을 1단부터 9단까지 Array 배열($9 \times 9 = 81$ 원소)에 저장하도록 프로그램을 완성하시오.

```
.data
Array BYTE 81 DUP(0)

.code
    MOV BL, 0 ;multiplicand(1~9)
    MOV ECX, 9
    MOV ESI, 0
L1:
    INC BL
    MOV BH, 0 ;multiplier(1~9)
    [ ]
    [ ]
L2:
    INC BH
    [ ]
    MUL BH ;multiplication
    MOV ARRAY[ESI], AL
    INC ESI
    LOOP L2
    [ ]
    LOOP L1
```

(2) [10pt] 부호없는 두 32비트 정수 VAR1과 VAR2의 곱을 SHIFT 연산을 통해 구하고자 한다. 결과가 EDX 레지스터에 저장되도록 프로그램을 완성하시오.

```
.data
VAR1 DWORD ?
VAR2 DWORD ?

.code
    MOV EBX, VAR2
    MOV ECX, 32
    ;to see each digit of VAR2
    MOV EDX, 0
L1:
    SHL EBX, 1
    [ ] ;if 0 jump to L2
    DEC ECX
    ;to compute power(2,ECX-1)
    MOV EAX, VAR1
    [ ] ;shift
    [ ]
    INC ECX
L2:
    LOOP L1
```

(3) [10pt] 다음 C/C++ 코드를 어셈블리 프로그램으로 작성하시오. (단, VAR1 ~ VAR5는 4byte int 형이라 가정함)

```
if ((VAR1 < VAR2) or ((VAR3 >= VAR4)))
    VAR5 = 1;
```

```
.data
VAR1 DWORD ?
VAR2 DWORD ?
VAR3 DWORD ?
VAR4 DWORD ?
VAR5 DWORD ?

.code
    MOV EAX, VAR1
    CMP EAX, VAR2
    [ ]
    MOV EAX, VAR3
    CMP EAX, VAR4
    [ ]
L1:
    MOV VAR5, 1
L2:
```

(5) [10pt] 다음 C/C++ 코드를 어셈블리 프로그램으로 작성하시오. (단, VAR1 ~ VAR4는 4byte int 형이라 가정함)

```
VAR4 = (4 * VAR1) % (VAR2 - VAR3))
```

```
.data
VAR1 DWORD ?
VAR2 DWORD ?
VAR3 DWORD ?
VAR4 DWORD ?
```

```
.code
    MOV EAX, VAR1
    SHL EAX, 2
    [ ]
    MOV EBX, VAR2
    [ ]
    [ ]
    [ ]
```